BÀI TẬP LÝ THUYẾT
TÌM KIẾM, CSP VÀ GAME TRONG AI

Họ và tên	Trương Văn Thiện
MSSV / Lớp	3123410356

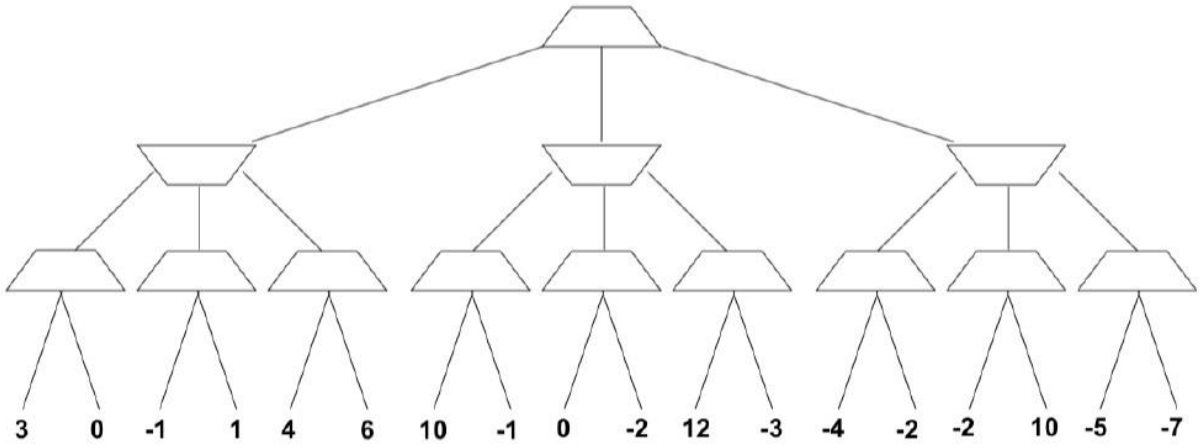
Mục tiêu học tập

- Thực hiện Minimax và Alpha-Beta Pruning, xác định giá trị các nút và các nhánh bị cắt tỉa.
- Phân tích cây trò chơi có nút ngẫu nhiên (chance nodes) và khả năng cắt tỉa.
- Vận dụng Forward Checking, Arc Consistency và suy luận trên đồ thị ràng buộc CSP.
- Giải thích tính đầy đủ, tối ưu của các thuật toán tìm kiếm và tác động khi chi phí cạnh thay đổi.
- Phân tích tác tử, khả năng quan sát của môi trường và tính hợp lý của hành vi.

Hướng dẫn: Trả lời các câu hỏi bằng lập luận ngắn gọn, trình bày các bước tính toán khi cần. Đối với câu có hình, có thể điền trực tiếp vào hình hoặc bổ sung bảng tính.

Bài 1. Minimax và Alpha-Beta Pruning

Xét cây trò chơi tổng bằng không trong Hình 1. Nút gốc là MAX, tầng tiếp theo là MIN và tầng kế tiếp là MAX. Hai người chơi đều hành động tối ưu. Các nút con được xét từ trái sang phải.



Hình 1. Cây trò chơi dùng cho Minimax và Alpha-Beta (nguồn: rec3.pdf, trang 1)

a) Áp dụng Minimax, tính giá trị mỗi nút trung gian trong cây.

- **Tầng 2 (Các nút MAX từ trái sang phải):**
  - $N_1 = \max(3,0) = 3$
  - $N_2 = \max(-1,1) = 1$
  - $N_3 = \max(4,6) = 6$
  - $N_4 = \max(10,-1) = 10$
  - $N_5 = \max(0,-2) = 0$
  - $N_6 = \max(12,-3) = 12$
  - $N_7 = \max(-4,-2) = -2$

- $N_8 = \max(-2,10) = 10$
- $N_9 = \max(-5,-7) = -5$

– **Tầng 1 (Các nút MIN từ trái sang phải):**
  - Nút MIN 1 (Trái): $\min(3,1,6) = 1$
  - Nút MIN 2 (Giữa): $\min(10,0,12) = 0$
  - Nút MIN 3 (Phải): $\min(-2,10,-5) = -5$
– **Gốc (Nút MAX):** Giá trị gốc $= \max(1,0,-5) = 1$

b) MAX nên chọn nhánh nào từ nút gốc? Giá trị trò chơi là bao nhiêu?

– Nhánh nên chọn: MAX nên chọn nhánh bên trái (dẫn đến nút MIN có giá trị 1).
– Giá trị của trò chơi: 1.

c) Áp dụng Alpha-Beta Pruning theo thứ tự trái sang phải. Ghi α, β ở các nút đã thăm; đánh dấu X các nhánh bị cắt tỉa.

Khởi tạo tại Gốc: $\alpha = -\infty, \beta = +\infty$. Duyệt từ trái sang phải:

1. Nhánh MIN 1 (Trái):
   - Nút MAX 1: thăm lá $3 \Rightarrow \alpha = 3$; thăm lá $0 \Rightarrow \alpha = 3$. Trả về 3.
   - MIN 1 cập nhật $\beta = \min(+\infty, 3) = 3$. $(\alpha = -\infty, \beta = 3)$
   - Nút MAX 2: thăm lá $-1 \Rightarrow \alpha = -1$; thăm lá $1 \Rightarrow \alpha = 1$. Trả về 1.
   - MIN 1 cập nhật $\beta = \min(3,1) = 1$. $(\alpha = -\infty, \beta = 1)$
   - Nút MAX 3: thăm lá $4 \Rightarrow \alpha = \max(-\infty, 4) = 4$.
     - Điều kiện cắt tỉa: $\alpha \geq \beta$ $(4 \geq 1) \Rightarrow$ Cắt tỉa lá 6
     - Nút MAX 3 trả về 4.
   - MIN 1 nhận giá trị 1.
   - Gốc (MAX) cập nhật $\alpha = \max(-\infty, 1) = 1$. $(\alpha = 1, \beta = +\infty)$
2. Nhánh MIN 2 (Giữa): truyền $(\alpha = 1, \beta = +\infty)$
   - Nút MAX 4: thăm lá $10 \Rightarrow \alpha = 10$; thăm lá $-1 \Rightarrow \alpha = 10$. Trả về 10.
   - MIN 2 cập nhật $\beta = \min(+\infty, 10) = 10$. $(\alpha = 1, \beta = 10)$
   - Nút MAX 5: thăm lá $0 \Rightarrow \alpha = 1$; thăm lá $-2 \Rightarrow \alpha = 1$. Trả về 0.
   - MIN 2 cập nhật $\beta = \min(10,0) = 0$.
     - Điều kiện cắt tỉa: $\alpha \geq \beta$ $(1 \geq 0) \Rightarrow$ Cắt tỉa toàn bộ nhánh nút MAX 6 (gồm các lá 12 và -3)!
   - MIN 2 trả về 0.

3. Nhánh MIN 3 (Phải): truyền ($\alpha = 1, \beta = +\infty$)
    - Nút MAX 7: thăm lá $-4 \Rightarrow \alpha = 1$; thăm lá $-2 \Rightarrow \alpha = 1$. Trả về -2.
    - MIN 3 cập nhật $\beta = \min(+\infty, -2) = -2$.
        - Điều kiện cắt tỉa: $\alpha \geq \beta$ $(1 \geq -2) \Rightarrow$ Cắt tỉa toàn bộ các nhánh còn lại (gồm nút MAX 8 chứa lá $-2,10$ và nút MAX 9 chứa lá $-5, -7$)

d) Liệt kê các nút lá không cần thăm. So sánh số nút lá được đánh giá bởi Minimax và Alpha-Beta.

– Các nút lá không cần thăm (bị cắt tỉa - 7 lá):

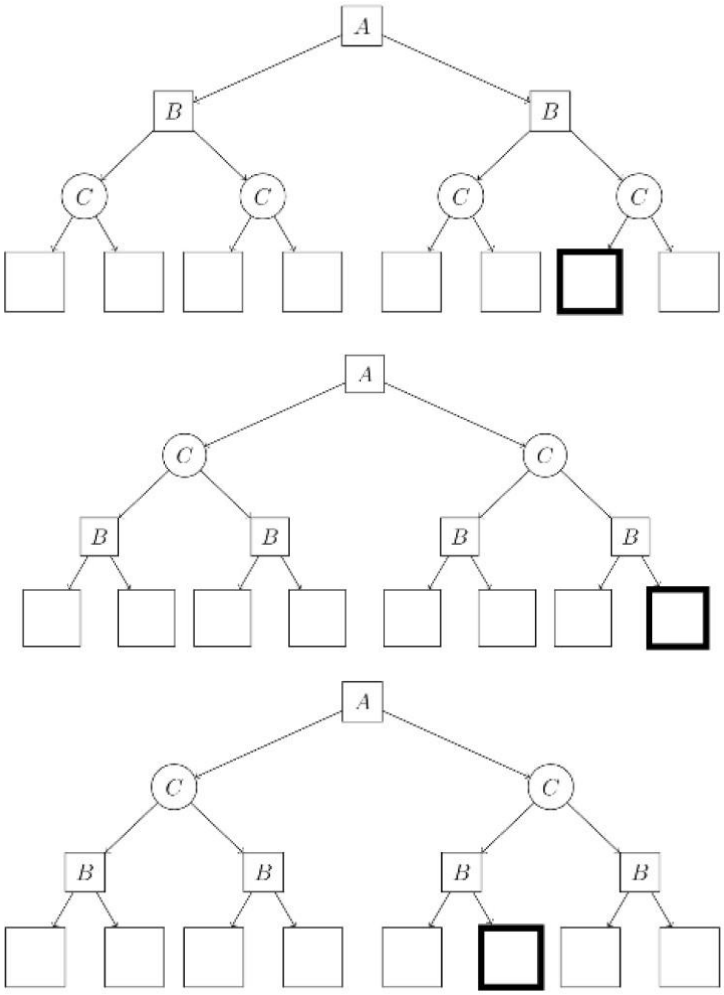
- Lá 6 (thuộc MAX 3)
- Lá 12 và -3 (thuộc MAX 6)
- Lá -2, 10 (thuộc MAX 8) và -5, -7 (thuộc MAX 9)

– So sánh:

- Minimax: Đánh giá tất cả 18 nút lá.
- Alpha-Beta: Chỉ đánh giá 11 nút lá (tiết kiệm 7 nút lá, tương đương giảm ~38.9% số lần duyệt lá).

Bài 2. Alpha-Beta trong cây có nút ngẫu nhiên (Expinimax)

Xét ba cây trò chơi ở Hình 2. A là nút MIN, B là nút MAX và C là nút ngẫu nhiên (chance node). Các con của C có xác suất bằng nhau. Duyệt từ trái sang phải. Với mỗi cây, hãy gán các giá trị hữu hạn cho các nút lá để có thể cắt tỉa nút được viền đậm, hoặc ghi "Không thể" và giải thích. Không thực hiện cắt tỉa trong trường hợp bằng nhau.



Hình 2. Ba trường hợp cắt tỉa trong cây Expinimax
(nguồn: rec3.pdf, trang 3)

Nút A là MIN, B là MAX, C là nút ngẫu nhiên (Chance node với xác suất các con là 0.5)

a) Trường hợp 1: điền các giá trị lá (nếu có thể) và chỉ ra điều kiện cắt tỉa.
Khả năng cắt tỉa: Có thể.

- Gán giá trị lá ví dụ:
  - Nhánh trái dưới A: gán các lá sao cho $V(B_{left}) = 10 \Rightarrow V(A) \leq 10$.
  - Nhánh C_1 dưới B_{right}: gán các lá có giá trị 15,15 $\Rightarrow V(C_1) = 15$.
- Điều kiện cắt tỉa: Do B là nút MAX nên $V(B_{right}) = \max(V(C_1), V(C_2)) \geq V(C_1) = 15$). Khi đó A (nút MIN) sẽ luôn chọn nhánh trái ($10 < 15$) bất kể giá trị $V(C_2)$ là bao nhiêu. Do đó nút viền đậm dưới C_2 được cắt tỉa

b) Trường hợp 2: điền các giá trị lá (nếu có thể) và chỉ ra điều kiện cắt tỉa.

- **Khả năng cắt tỉa:** Có thể. (khi miền giá trị lá hữu hạn và có biên).
- **Gán giá trị lá ví dụ:**
  - Cây bên trái: tất cả lá bằng $10 \Rightarrow V(C_{left}) = 10$.
  - Ở cây bên phải C_{right}: nút B_1 có các lá bằng $30 \Rightarrow V(B_1) = 30$.
- **Điều kiện cắt tỉa:** Giá trị $V(C_{right}) = 0.5 \times 30 + 0.5 \times V(B_2) = 15 + 0.5 \times V(B_2)$. Nếu các lá không âm ($\geq 0$), $V(C_{right}) \geq 15 > 10$. Nút MIN A chắc chắn chọn nhánh trái (10), nên nút viền đậm dưới B_2 được cắt tỉa.

c) Trường hợp 3: điền các giá trị lá (nếu có thể) và chỉ ra điều kiện cắt tỉa.

- Khả năng cắt tỉa: Có thể.
- Gán giá trị lá ví dụ: Giống trường hợp 2, gán cây trái có giá trị 10, nút B_1 ở nhánh phải có giá trị 30.
- Điều kiện cắt tỉa: Vì $V(C_{right}) \geq 15 > 10$ ngay từ khi chỉ xét B_1, toàn bộ nút B_2 (bao gồm cả lá viền đậm) không ảnh hưởng tới quyết định của A nên được cắt tỉa

d) Giải thích vì sao cắt tỉa trong cây có nút ngẫu nhiên khác với Alpha-Beta trong cây Minimax.

Trong cây Minimax chuẩn, các nút MIN/MAX chỉ lấy giá trị chằn chặn min / max, tạo ra các giới hạn cứng (α, β) để cắt tỉa triệt để. Trong cây Expectimax, nút ngẫu nhiên lấy trung bình trọng số ($\sum p_i v_i$). Một nút con cực kỳ xấu hoặc cực kỳ tốt có thể làm thay đổi hoàn toàn giá trị kỳ vọng. Vì vậy, trừ khi có giới hạn miền giá trị hữu hạn (bounded utility), việc cắt tỉa trong cây ngẫu nhiên khó thực hiện hơn rất nhiều.

Bài 3. Câu hỏi đúng/sai

Xét cây trò chơi có gốc MAX, giá trị Minimax là V_M. Thay mọi nút MIN bằng nút ngẫu nhiên có phân phối xác suất bất kỳ nhưng đã biết, ta nhận cây có giá trị Expectimax là V_E. Với mỗi mệnh đề, chọn Đúng/Sai và giải thích.

a) Luôn có $V_M \leq V_E$. ☑ Đúng ☐ Sai

Nút MIN chọn hành động tối thiểu hóa tuyệt đối $\min(x_i)$. Nút ngẫu nhiên tính trung bình kỳ vọng $\sum p_i x_i$. Vì $\min(x_i) \leq \sum p_i x_i$ với mọi phân phối xác suất, nên $V_M \leq V_E$.

b) Dùng chính sách Minimax tối ưu ở cây đã thay nút MIN bằng nút ngẫu nhiên sẽ đảm bảo lợi ích kỳ vọng ít nhất là V_M. ☑ Đúng ☐ Sai

Chính sách Minimax được thiết kế an toàn nhất để chống lại đối thủ tối ưu. Khi đối thủ hành động ngẫu nhiên (không đối kháng hoàn hảo), kết quả chỉ có thể bằng hoặc tốt hơn trường hợp xấu nhất V_M.

c) Dùng chính sách Minimax tối ưu ở cây đã thay nút MIN bằng nút ngẫu nhiên sẽ đảm bảo lợi ích kỳ vọng ít nhất là V_E. ☐ Đúng ☑ Sai

V_E đạt được khi dùng chính sách Expectimax tối ưu (khai thác sự ngẫu nhiên). Chính sách Minimax chọn nước đi an toàn hơn nên lợi ích kỳ vọng thu được chỉ đảm bảo $\geq V_M$, không đảm bảo đạt tới V_E.

Xét trò chơi một người, kết quả hành động có thể không xác định trước. Người chơi nhận +1 khi thắng và −1 khi thua. Đánh dấu mọi phương pháp có thể phù hợp để mô hình hóa và giải trò chơi theo giả định cần thiết.

☐ Minimax với tiện ích trạng thái cuối +1/−1

☑ Expectimax với tiện ích trạng thái cuối +1/−1

☑ Value Iteration với phần thưởng 0 ở trạng thái trung gian và +1/−1 ở kết quả thắng/thua

☐ Không phương án nào

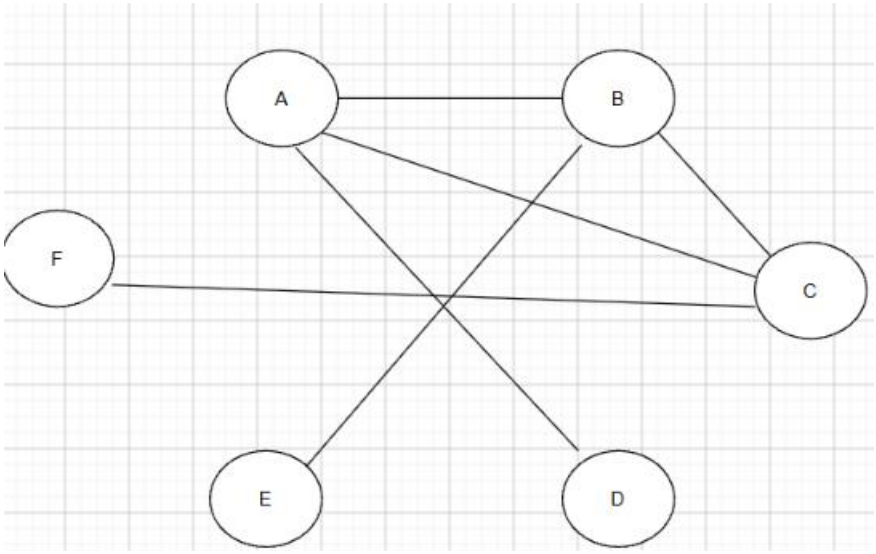
d) Giải thích ngắn gọn lựa chọn ở trên và những giả định cần thiết.

- Expectimax: Phù hợp vì trò chơi 1 người có yếu tố ngẫu nhiên (nút ngẫu nhiên mô hình hóa kết quả không xác định). Giả định: Cây trò chơi có độ sâu hữu hạn và xác suất chuyển trạng thái đã biết.
- Value Iteration: Phù hợp vì trò chơi mô hình hóa thành MDP (Quá trình quyết định Markov). Giả định: Không gian trạng thái hữu hạn và xác suất chuyển trạng thái $P(s'|s,a)$ đã biết.

Bài 4. CSP với Constraint Graph và Lan truyền ràng buộc

Có 6 sinh viên A, B, C, D, E, F cần sắp chỗ ngồi. Các cặp có ràng buộc nhị phân: (A,B), (A,C), (A,D), (B,C), (B,E), (C,F).

a) Vẽ đồ thị ràng buộc tương ứng (mỗi biến là một đỉnh; mỗi ràng buộc nhị phân là một cạnh).



b) Sau khi gán giá trị cho A, miền của những biến nào có thể bị thay đổi bởi Forward Checking?

FC chỉ kiểm tra các biến láng giềng trực tiếp với biến vừa gán (A).
Các biến có thể bị thay đổi miền giá trị: B, C, D.

c) Sau khi gán A và thực hiện Forward Checking, tiếp tục gán B: miền của những biến nào có thể bị thay đổi khi Forward Checking tại B?

FC tại B chỉ kiểm tra các biến chưa gán là láng giềng trực tiếp của B.

Các biến có thể bị thay đổi miền giá trị: C, E

d) Sau khi gán A, nếu duy trì Arc Consistency, miền của những biến nào có thể thay đổi?

AC lan truyền ràng buộc trên toàn bộ đồ thị liên thông.

Các biến có thể bị thay đổi miền giá trị: B, C, D, E, F (tất cả các biến chưa gán).

e) Sau khi gán A và duy trì Arc Consistency, tiếp tục gán B và duy trì Arc Consistency: miền của những biến nào có thể thay đổi?

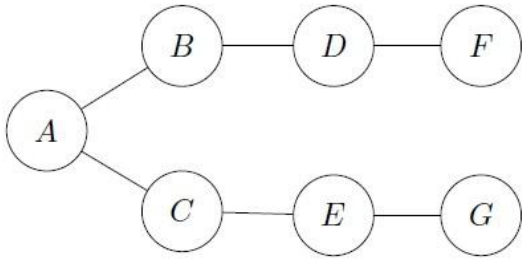
- Các biến có thể bị thay đổi miền giá trị: C, D, E, F (tất cả các biến chưa gán còn lại).

f, g, h) Phân tích số lần backtrack trong trường hợp xấu nhất (Hình 3)

Đồ thị Hình 3 có cấu trúc CÂY (Tree-structured CSP) chia làm 2 nhánh từ A:

- Nhánh 1: $A - B - D - F$
- Nhánh 2: $A - C - E - G$

Xét thuật toán mới: thực hiện Arc Consistency ban đầu và sau mỗi lần đã gán một số chẵn biến (2, 4, 6, ...). Mỗi biến có miền kích thước d. Sử dụng đồ thị ở Hình 3 để phân tích số lần backtrack trong trường hợp xấu nhất cho từng thứ tự gán; nêu rõ cách tính và các giả định.



th ràng bu c cho bài toán th t gán bi n (ngu n: rec3.pdf, trang 6)

f) Thứ tự A → B → C → D → E → F → G

Số lần backtrack xấu nhất = 0.

Giải thích: Thuật toán chạy AC ở bước 0 (trước khi gán). Đối với CSP dạng cây, chạy Arc Consistency ban đầu và gán biến theo thứ tự từ gốc đến lá (topological order) đảm bảo tìm ra lời giải mà không xảy ra quay lui (backtrack-free).

g) Thứ tự G → E → C → A → B → D → F

Số lần backtrack xấu nhất = 0.

Giải thích: AC được thực hiện định kỳ ở các bước gán chẵn (bước 2 khi gán xong E, bước 4 khi gán xong A, bước 6). Việc thực hiện AC định kỳ loại bỏ hoàn toàn các giá trị không hợp lệ trước khi xảy ra xung đột.

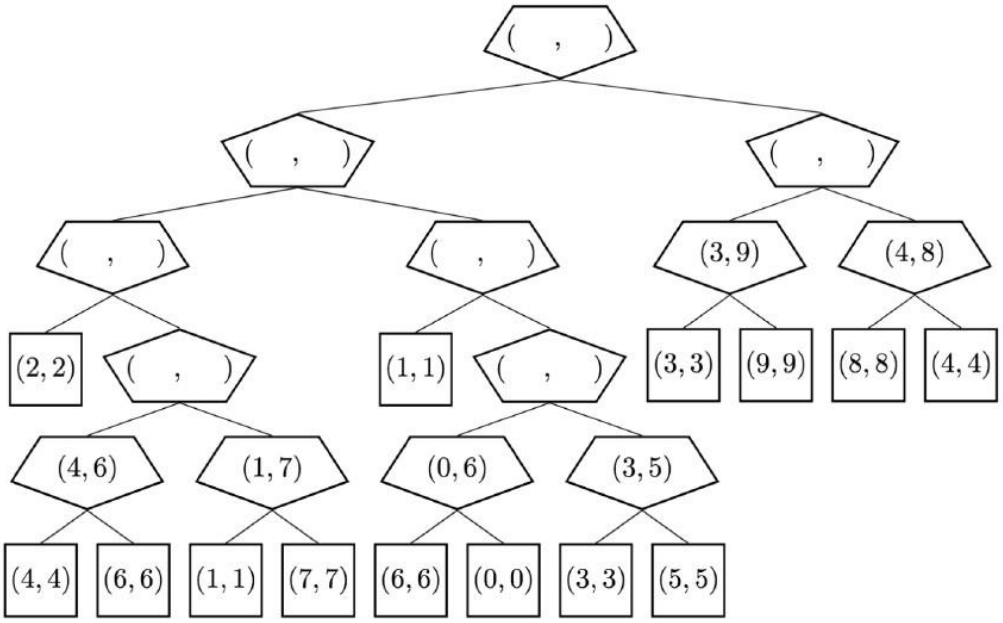
h) Thứ tự G → F → E → D → C → B → A

Số lần backtrack xấu nhất = 0.

Giải thích: Nhờ AC thực hiện ban đầu và cập nhật ở mọi bước gán số chẵn trên đồ thị dạng cây, mọi nhánh cụt đều được phát hiện ngay lập tức và loại bỏ, không dẫn đến backtrack.

Bài 5. Cây trò chơi Pacman và Casper

Pacman tìm cách tối đa hóa tiện ích. Casper là một bóng ma thân thiện, thực sự muốn tăng tiện ích cho Pacman nhưng Pacman tưởng Casper cố giảm tiện ích. Casper biết Pacman đang hiểu nhầm. Ở mỗi nút, cặp (u, v) biểu thị lần lượt giá trị mà Pacman tin tưởng và giá trị theo hành vi thực tế của Casper. Tại lá, u = v = tiện ích cuối.



Casper

a) Điền tất cả cặp (u, v) còn thiếu trong Hình 4. Giải thích sự khác biệt giữa u và v tại nút Casper.

1. Các nút Pacman ở tầng 2 (từ trái sang phải):
   - Nút 1 (trên lá $(2,2)$ và nút $(4,6)$): chọn nhánh phải $(4 > 2) \Rightarrow (4,6)$
   - Nút 2 (trên nút $(0,6)$ và $(3,5)$): chọn nhánh phải $(3 > 0) \Rightarrow (3,5)$
   - Nút 3 (trên lá $(1,1)$ và nút $(3,5)$): chọn nhánh phải $(3 > 1) \Rightarrow (3,5)$
   - Nút 4 (trên nút $(3,9)$ và $(4,8)$): chọn nhánh phải $(4 > 3) \Rightarrow (4,8)$
2. Các nút Casper ở tầng 1:
   - Nút Casper Trái (có 2 con Pacman là $(4,6)$ và $(3,5)$):
     - $u = \min(4,3) = 3$
     - Casper chọn nhánh có v lớn hơn $(6 > 5) \Rightarrow v = 6$
     - Giá trị cặp: $(3,6)$
   - Nút Casper Phải: Giá trị cặp là $(4,8)$
3. Nút Gốc (Pacman):
   - So sánh u của 2 nút con: Left $(3,6)$ vs Right $(4,8)$.
   - Pacman chọn nhánh phải vì $4 > 3$.
   - Giá trị cặp tại gốc: $u = \max(3,4) = 4$, $v = v_{right} = 8 \Rightarrow (4,8)$.

Giải thích sự khác biệt giữa u và v tại nút Casper: u là giá trị mà Pacman giả định Casper sẽ hạ thấp nhất có thể, còn v là giá trị thực tế Casper chủ động chọn để hỗ trợ Pacman.

b) Theo thứ tự duyệt trái sang phải, đánh dấu các nhánh có thể không cần duyệt khi Casper tính giá trị trò chơi.

Khi Casper đánh giá các nhánh, nếu đã biết một nhánh đem lại giá trị thực tế v cao hơn cho Pacman và khiến Pacman chọn nước đi có lợi, các nhánh có giá trị v kém hơn mà không ảnh hưởng tới u của Pacman có thể không cần duyệt sâu.

c) Nếu Pacman biết Casper thân thiện thì giá trị toàn bộ cây thay đổi như thế nào?

Nếu Pacman hiểu đúng Casper, trò chơi trở thành trò chơi hợp tác thuần túy. Cả hai đều chọn nước đi tối đa hóa tiện ích.

- Giá trị toàn bộ cây = $\max(\text{tất cả các lá}) = \max(4,6,1,7,6,0,3,5,3,9,8,4) = 9$.
- Giá trị thực tế mà Pacman nhận được tăng từ 8 lên 9.

Bài 6. Thuật toán tìm kiếm – Tính đầy đủ và tối ưu

Với mỗi thuật toán sau, xác định điều kiện để được đảm bảo đầy đủ (Complete) và tối ưu (Optimal). Phân biệt tìm kiếm trên cây và trên đồ thị nếu cần.

Thuật toán	**Điều kiện đầy đủ**	**Điều kiện tối ưu**
Breadth-First Search (BFS)	Hệ số nhánh b hữu hạn.	Chi phí mọi bước bằng nhau ($c = \text{const} > 0$).
Depth-First Search (DFS)	Tree search: Không đầy đủ nếu không gian vô hạn/có vòng lặp.	Không tối ưu.
Iterative Deepening Search (IDS)	Hệ số nhánh b hữu hạn.	Chi phí mọi bước bằng nhau ($c = \text{const} > 0$).
A* Search	Hệ số nhánh b hữu hạn và chi phí mỗi bước $c \geq \epsilon > 0$.	Tree search: Heuristic $h(n)$ phải Admissible ($0 \leq h(n) \leq h^*(n)$). Graph search: Heuristic

		$h(n)$ phải Consistent ($h(n) \leq c(n, a, n') + h(n')$).

Giải thích vì sao điều kiện chi phí bước và đặc tính heuristic có thể ảnh hưởng đến kết luận.

- Chi phí bước $\epsilon > 0$: Tránh trường hợp thuật toán bị mắc kẹt trong đường đi vô hạn có tổng chi phí không tăng (Zeno's paradox).
- Đặc tính Heuristic: Tính Admissible/Consistent đảm bảo $f(n) = g(n) + h(n)$ không bao giờ đánh giá thấp chi phí thực tế, giúp A^* không mở rộng nhầm đường đi không tối ưu trước khi tìm ra nghiệm chuẩn.

Bài 7. A* khi chi phí một cạnh thay đổi

Sau khi A* graph search tìm được đường đi tối ưu từ trạng thái đầu đến đích, chi phí cạnh X → Y thay đổi từ c thành c + n hoặc c − n, với n > 0. Với từng tình huống, hãy phân tích liệu chi phí tối ưu có thay đổi và cách cập nhật/tìm đường tối ưu mới thay vì nhất thiết chạy lại từ đầu. Khi giảm chi phí, giả sử chi phí mới vẫn hợp lệ theo mô hình tìm kiếm.

a) Chi phí tăng n; cạnh X → Y thuộc đường tối ưu cũ và đã được khám phá.

- Chi phí tối ưu thay đổi? Có thể tăng (tối đa thành $C^* + n$) hoặc giữ nguyên nếu có đường đi khác thay thế.
- Cập nhật: Cập nhật g(Y) = g(X) + (c + n) và lan truyền cho các nút con của Y. Tiếp tục chạy A^* từ tập OPEN hiện tại mà không cần chạy lại từ đầu.

b) Chi phí giảm n; cạnh X → Y thuộc đường tối ưu cũ và đã được khám phá.

- Chi phí tối ưu thay đổi? Có, giảm đúng n thành $C^* - n$.
- Cập nhật: Đường đi cũ chắc chắn vẫn tối ưu. Chỉ cần giảm chi phí tối ưu xuống $C^* - n$, không cần chạy lại A^*.

c) Chi phí tăng n; cạnh X → Y không thuộc đường tối ưu cũ nhưng đã được khám phá.

- Chi phí tối ưu thay đổi? Không.
- Cập nhật: Tăng chi phí của một cạnh không nằm trên đường đi tốt nhất càng làm nó kém hấp dẫn hơn. Giữ nguyên đường tối ưu cũ

d) Chi phí giảm n; cạnh X → Y không thuộc đường tối ưu cũ nhưng đã được khám phá.

- Chi phí tối ưu thay đổi? Có thể (nếu đường đi mới qua X → Y có giá trị $< C^*$).

- Cập nhật: Cập nhật $g(Y) = \min(g(Y), g(X) + c_{new})$). Đưa Y vào lại tập OPEN với f(Y) mới và tiếp tục chạy A*.

e) Chi phí tăng n; cạnh X → Y không thuộc đường tối ưu cũ và chưa được khám phá.

- Chi phí tối ưu thay đổi? Không.
- Cập nhật: Giữ nguyên đường đi tối ưu cũ, không cần thực hiện thêm thao tác nào.

f) Chi phí giảm n; cạnh X → Y không thuộc đường tối ưu cũ và chưa được khám phá.

- Chi phí tối ưu thay đổi? Có thể.
- Cập nhật: Cập nhật chi phí mới của cạnh X → Y. Nếu $g(X) + (c - n) + h(Y) < C^*$, thêm Y vào tập OPEN và tiếp tục chạy A*.

Bài 8. Tác tử thông minh và môi trường

Xét ba kiểu tác tử, trong đó các hàm f_A, f_B, f_C có thể ngẫu nhiên nhưng không lưu trạng thái nội bộ:

• A: act(percept) → f_A()

• B: act(percept) → f_B(percept)

• C: lưu danh sách percepts; mỗi lần act(percept) thêm percept rồi trả về f_C(percepts)

Với mỗi đặc tính môi trường sau, hãy liệt kê các lớp tác tử có thể tồn tại một hàm f để hành động hợp lý hoàn hảo trong mọi môi trường thuộc lớp đó. Giải thích ngắn gọn.

a) Quan sát đầy đủ; xác định; rời rạc; một tác tử; tĩnh.

- Tác tử phù hợp: B và C
- Giải thích: Môi trường quan sát đầy đủ nên percept hiện tại chứa toàn bộ thông tin trạng thái. Hàm $f_B(percept)$ ánh xạ trực tiếp trạng thái hiện tại sang hành động tối ưu là đủ để hành động hợp lý hoàn hảo.

b) Quan sát một phần; xác định; rời rạc; một tác tử; tĩnh.

- Tác tử phù hợp: Chỉ C
- Giải thích: Quan sát một phần khiến nhận thức đơn lẻ không phản ánh đủ trạng thái thực. Tác tử C lưu giữ toàn bộ lịch sử percepts để khôi phục trạng thái niềm tin (belief state) và đưa ra quyết định chính xác.

c) Quan sát một phần; ngẫu nhiên; rời rạc; một tác tử; tĩnh.

- Tác tử phù hợp: Chỉ C

- Giải thích: Yếu tố ngẫu nhiên kết hợp quan sát một phần bắt buộc phải duy trì phân phối xác suất trên các trạng thái dựa vào toàn bộ lịch sử quan sát để tối đa hóa lợi ích kỳ vọng.

d) Đúng hay sai: Có tồn tại một môi trường mà mọi tác tử đều hành động hợp lý. Nêu ví dụ hoặc phản biện.

Đúng.

Ví dụ: Xét một môi trường mà mọi hành động đều đem lại cùng một mức thưởng/tiện ích như nhau (ví dụ: môi trường không có phạt, thưởng luôn bằng 0 bất kể tác tử làm gì). Khi đó, bất kỳ hàm $f_A()$, $f_B(\text{percept})$ hay $f_C(\text{percepts})$ nào cũng đều mang lại kết quả tối ưu, tức là mọi tác tử A, B, C đều hành động hợp lý hoàn hảo.

— Hết —